



Events. Experiences. Together.

ADMIN PORTAL (INTERNAL / BACK-OFFICE)

Event Platform Management Module

Technical Development Document

Version 1.0

Prepared for: Zordr Food Private Limited

Date: August 30, 2026

Scope: Internal Admin Web Portal — 16-screen UI reference (Login, Dashboard, Organizers, Events, Orders, Customers, Settlements, Refunds, Support, Analytics, Employees, Roles, Settings)

1. Introduction & Purpose

This document specifies the technical design and development plan for the Zordr Admin Portal — the internal back-office application used by Zordr's own team to operate the marketplace: approving organizers, moderating events before they go live, overseeing orders/settlements/refunds, running customer support, tracking platform analytics, and managing internal employee access.

It is derived from the 16-screen UI reference set (Login, Dashboard, Organizers list + detail, Events list + Review, Orders, Customers, Settlements, Refunds, Support, Analytics, Employees, Manage Roles, Create Role, Settings) and translates that UI into functional behaviour, data model, technology stack, architecture, API design, and business/validation rules.

This is the third and final leg of the Zordr platform alongside the Organizer Portal and the Customer App. It is the control-plane that sits above both: organizer onboarding submitted in the Organizer Portal is approved here; events published by organizers pass through a moderation gate here before becoming visible in the Customer App; and orders/settlements/refunds generated by the other two modules are all reconciled and, where needed, overridden here.

2. Scope & Module Breakdown

#	Module	Screens	Purpose
1	Admin Login	1	Email/password authentication for internal staff, monitored access
2	Dashboard	1	Platform-wide KPIs, trends, recent activity, live events, quick actions
3	Organizers	2	Organizer directory + KYC approval, detail view with events/revenue/settlements/support/notes
4	Events	2	Platform-wide event directory + a moderation Review workflow (Approve/Send Back/Reject) gating publication
5	Orders	1	Every ticket order across all organizers/events, payment/order status, bulk actions
6	Customers	1	All registered attendees, spend/activity, block/unblock
7	Settlements	1	Platform-wide payout ledger, fee breakdown, status, report generation
8	Refunds	1	Refund request queue, review/approve/reject
9	Support	1	Cross-platform ticketing (organizer + customer issues), SLA tracking
10	Analytics	1	GMV/order/customer trends, top performers, breakdowns
11	Employees & Roles	3	Internal staff directory, RBAC role management, role creation
12	Settings	1	Platform org profile, preferences, plan, danger zone

Out of scope for this document: the Organizer Portal and Customer App (each covered in their own technical documents) — referenced here only at their integration points.

3. User Roles & Access Control (Internal RBAC)

The Admin Portal is itself the source of truth for internal role-based access, as shown in the Employees/Manage Roles/Create Role screens. Confirmed system roles:

Role	Scope (as configured)	Typical Modules
Super Admin	Full access to all modules and settings	All — Dashboard, Organizers, Events, Orders, Customers, Settlements, Refunds, Support, Analytics, Settings, Employees
Event Manager	Manage events and organizers	Dashboard, Events, Organizers
Operations Manager	Handle orders, settlements and refunds	Orders, Settlements, Refunds
Support Executive	Manage customer support tickets	Support (+ read access to Orders/Customers as needed)
Finance Executive	Access to settlements, refunds and financial reports	Settlements, Refunds, Analytics (financial)
Marketing Executive	Access to events, analytics and customer data	Events (read), Analytics, Customers (read)
Custom Roles	Org-defined, built via Create Role with a per-module View/Create/Edit/Delete/Export matrix	Configurable

Tech rule: system roles (Super Admin, Event Manager, etc.) cannot be deleted, only have their permissions modified, per the explicit notice on the Manage Roles screen; only Custom Roles are fully deletable. Every module-level action (View/Create/Edit/Delete/Export) must be enforced by an authorization guard keyed off the acting employee's role — the Create Role screen's permission grid is the literal schema for this guard.

4. Functional Specification by Screen

4.1 Admin Login

- Email + password only (no OTP/SSO, consistent with a small, tightly controlled internal user base).
- Explicit "Authorized personnel only — all access is monitored and logged" notice — implies mandatory audit logging of every login and, ideally, every privileged action taken afterward.

Tech rule: failed login attempts should be rate-limited/lockout-protected; every successful login should write an AuditLog entry (actor, IP, device, timestamp) given the stated monitoring posture.

4.2 Dashboard

- KPI cards: Today's GMV, Today's Registrations, Active Events, Pending Approvals (events & organizers combined), Failed Payments, Refund Requests, Live Events, Pending Settlements — each with a day-over-day delta.
- Trend charts: Daily Revenue (GMV), Registrations, Platform Growth (Events vs Customers) — selectable time range.
- Recent Activity — a unified, platform-wide event log (organizer onboarded, event created, ticket sold, refund requested, settlement completed, event published, support ticket, payment failed) with type, detail, user, and status.
- Live Events ticker — currently-running events with live attendance counts.
- Quick Actions — Approve Organizer, Approve Event, Issue Refund, Create Admin.

Tech rule: this dashboard aggregates across the entire platform (all organizers/events), unlike the Organizer Portal's dashboard which is scoped to one organizer — it must be backed by a platform-wide aggregation/reporting service (e.g. a data warehouse or materialized views), not a naive join across live transactional tables, to stay performant at scale.

4.3 Organizers — List & Detail

- List: KPI strip (Total/Approved/Pending/Rejected/Blocked), searchable/filterable table (status, KYC status, city) with per-organizer active events and total revenue, row actions.
- Detail — tabs: Overview, Documents, Bank & GST, Events, Revenue, Settlements, Support Tickets, Notes.
- Overview shows org identity, primary contact, org info, lifetime stats, recent events/settlements/support tickets.
- Actions: Update Organizer, Send Message, Suspend Organizer, Reject Organizer, Reset Password, and an internal Quick Notes / Recent Notes log.

Tech rule: this is the approval authority for the Organizer Portal's onboarding submission (Organizer Portal §4.2) — an organizer's KYC/Approved status is written here, not self-service; Suspend/Reject must immediately cascade to block that organizer's ability to publish/edit events and receive payouts. All admin notes and status changes are timestamped and attributed to the acting employee (audit trail).

4.4 Events — List & Review

- List: KPI strip (Total/Published/Pending Review/Completed/Cancelled), filterable table with a distinct "Pending Review" status row action ("Review") versus "View" for already-published events.
- Event Review screen: full read-only render of the organizer's submitted event (banner/gallery, description, venue+map, schedule, tickets & pricing, organizer info, policies, terms) alongside a Review Checklist (banner & images / details & description / date-time-venue / ticket types & pricing / organizer info / policies & terms / content guidelines) that must be fully checked.
- Actions: Preview as Customer, Send Back (to organizer with notes), Reject, Approve & Publish; an Admin Notes field and a Previous Review Activity audit trail (submitted for review, event created, by whom, when).

Tech rule: this is the moderation gate implied but not visible in the Organizer Portal's own Publish step (Organizer Portal §4.5 Step 7) — an event's true lifecycle is therefore Draft → Submitted (by organizer) → Pending Review (admin queue) → Published (admin-approved) / Sent Back / Rejected, not simply Draft → Published as the organizer-side document alone would suggest. The Customer App's discovery/search must only ever surface events in the Published state. Send Back should reopen the event for organizer editing while preserving the submitted version for reference/diff.

4.5 Orders

- KPI strip (Total/Paid/Pending/Failed/Refunded Orders with success rate), rich filter set (event, organizer, payment status, order status, date range), Export and Bulk Actions.
- Table: Order ID, Customer, Event, Organizer, Amount, Tickets, Payment Gateway, Payment Status, Order Status, Created At.

Tech rule: this view is a read/administrative-action layer over the same Order/Ticket entities used by the Customer App and reported in the Organizer Portal's Settlements — it must never be a separate copy of order data. Bulk Actions (e.g. bulk refund, bulk export) must be queued/async for large selections and fully audit-logged given the financial sensitivity.

4.6 Customers

- KPI strip (Total/Active/New This Month/Repeat/Banned-Blocked Customers), searchable/filterable table with Events Attended, Lifetime Spend, Last Order, Status, Joined On.
- Row actions presumably include block/unblock and view detail (mirrored pattern from Organizers).

Tech rule: 'Events Attended' and 'Lifetime Spend' must be computed the same way as in the Customer App's own Profile screen (CheckIn-derived attendance, captured-order-derived spend) so the two surfaces never disagree; Blocked customers must be prevented from completing new checkouts platform-wide, not just flagged for display.

4.7 Settlements

- KPI strip (Total/Paid/Pending/On Hold Settlement Amounts), tabs (All/Pending/Paid/On Hold/Failed), filterable table: Organizer, Event(s), Settlement Period, Gross Sales, Platform Fee (shown here as 5%), Gateway Fee, Refunds, Net Payout, Status, Transfer Date.
- Generate Settlement and Download Report actions; a Settlement Policy footer ("payouts initiated within 5–7 business days from end of settlement period, subject to no pending refunds/disputes").

Tech rule / reconciliation flag: the Platform Fee is displayed here as 5%, while the Organizer Portal's Create-Event Tickets step discloses "2% + ₹5/ticket" to organizers. These must be reconciled into a single authoritative fee-configuration source before development — either the two are genuinely different fee lines (e.g. a 2% payment-processing pass-through disclosed to organizers at checkout time vs. a blended 5% platform commission booked internally) or one figure is stale; whichever it is, both the Organizer Portal and this screen must read from the same FeeConfig entity, not independently hardcoded values. Settlement generation should remain a scheduled, idempotent batch process (as specified in the Organizer Portal document §9.5), with this Admin screen providing oversight/override (Generate, Hold, mark Failed) rather than ad hoc manual computation.

4.8 Refunds

- KPI strip (Total/Processed/Pending/Rejected Refunds), filterable table: Refund ID, Order ID, Customer, Event, Amount, Reason, Status, Requested On, Processed On, with a "Review" action for Pending items and "View" for resolved ones.
- Refund Policy footer (5–7 business days processing after approval).

Tech rule: approving a refund here must trigger the same automated gateway-refund + notification workflow described for organizer-initiated cancellations (Organizer Portal §9.6) and must correspondingly adjust the relevant Settlement's Net Payout if the refund lands within an open settlement period.

4.9 Support

- KPI strip (Total/Resolved/Open Tickets, Avg. Response Time), filterable table across both organizer and customer-originated tickets, with Category, Event, Status, Priority.
- Sidebar: Ticket Categories breakdown, Quick Actions (Create Ticket, View FAQs, Support Inbox, Live Chat), and Support SLA panel (first response < 6 hours, resolution < 24 hours, CSAT 4.7/5).

Tech rule: SLA timers (first response, resolution) must be computed and monitored server-side against ticket-created and first-reply timestamps, with alerting when a ticket is at risk of breaching SLA — this is an operational monitoring requirement, not just a display metric.

4.10 Analytics

- Platform-wide KPI strip (Total GMV, Orders, Tickets Sold, Active Events, Unique Customers, Conversion Rate).
- Charts: GMV Trend, Orders vs Customers, Revenue by Payment Method (donut), Order Status Distribution (donut), Settlement Overview (donut), User Growth.
- Ranked tables: Top Events by Revenue, Top Organizers by GMV, Customers by City.
- Download Full Report action and a narrative Insight callout.

Tech rule: given the volume of cross-cutting aggregation (by event, organizer, city, payment method, time), this screen should be backed by a dedicated analytics/OLAP layer (e.g. scheduled ETL into a reporting schema or warehouse) rather than live queries against the transactional Postgres tables also serving checkout traffic, to avoid analytics load degrading production performance.

4.11 Employees, Manage Roles, Create Role

- Employees: KPI strip (Total/Active/Inactive Employees), table (Name, Email, Role, Department, Status, Last Active), inline "Add New Employee" panel (name, email, role, department, send-invitation toggle) with a live Role Permissions Preview.
- Manage Roles: role list (system + custom) with employee counts, and a permission matrix (View/Create/Edit/Delete/Export) per module for the selected role, editable and savable.

- Create Role: role name/description/type (System vs Custom Role), full module permission grid with Select Preset / Select All / Deselect All, live Quick Preview and Permission Summary.

Tech rule: the permission matrix here (module × View/Create/Edit/Delete/Export) is the literal authorization schema the entire Admin Portal's API must enforce — every admin-facing endpoint should check the acting employee's role-permission grid for (module, action) before proceeding, ideally via a shared policy/guard layer rather than per-endpoint ad hoc checks. New employee invitations should issue a time-limited, single-use invite token (not a shared/default password).

4.12 Settings

- Tabs: General, Users & Access, Notifications, Payments, Platform, Security, Integrations.
- General: platform organization details (name, support email, contact, address, logo), Preferences (timezone, date/time format, currency, language, items-per-page), Platform Information (plan, member since, events created, total users, storage used), Recent Activity log, and a Danger Zone (Delete Organization).

Tech rule: 'Delete Organization' here refers to the Zordr platform tenant itself (catastrophic, presumably reserved for a multi-tenant SaaS future or a decommissioning path) — it must be gated far more strictly than any other destructive action in the system (e.g. Super-Admin-only, secondary confirmation, possibly requiring Zordr leadership sign-off outside the UI itself).

5. Data Model — Additions Over the Organizer/Customer Schema

The Admin Portal reads and administers the same core entities defined in the Organizer Portal (\$5) and Customer App (\$5) documents — Organizer, Event, TicketType, Order, Ticket, Settlement, Registration, CheckIn, User — with these additions specific to internal operations:

Entity	Purpose	Key Fields
AdminEmployee	Internal staff account	id, name, email, role_id, department, status(active/inactive), last_active_at
Role	System or custom internal role	id, name, description, type(system/custom), is_deletable
RolePermission	Per-module CRUD+export grants for a role	role_id, module, can_view, can_create, can_edit, can_delete, can_export
EventReview	Moderation record for an event submission	event_id, status(pending/approved/sent_back/rejected), checklist(JSON), admin_notes, reviewed_by, reviewed_at
OrganizerNote	Internal notes on an organizer	organizer_id, author_id, note, created_at
AuditLog	Every privileged admin action	actor_id, action, entity_type, entity_id, before(JSON), after(JSON), ip, created_at
SupportTicket	Cross-platform support ticket (organizer or customer)	id, requester_type, requester_id, category, event_id, priority, status, created_at, resolved_at
FeeConfig	Authoritative platform fee schedule	id, platform_fee_pct, per_ticket_fee, gateway_fee_pct, effective_from

FeeConfig is called out specifically to resolve the 2%-vs-5% platform fee discrepancy noted in §4.7 — both the Organizer Portal's ticket-pricing disclosure and this Admin Portal's Settlements computation must read from this single table.

6. Recommended Technology Stack

Shared platform/backend with the Organizer Portal and Customer App wherever possible; the Admin Portal is a separate, internally-restricted frontend.

Frontend

Layer	Choice	Why
Framework	Next.js 14+ (React 18, TypeScript), desktop-first	Data-dense tables/dashboards benefit from a wide desktop layout; shares component library with the Organizer Portal
Styling / UI Kit	Tailwind CSS + shadcn/ui + Radix	Consistent Zordr design system; shadcn's data-table primitives suit the heavy tabular UI throughout
Tables & Data Grids	TanStack Table	Sorting/filtering/pagination for Organizers, Events, Orders, Customers, Settlements, Refunds, Support lists
Forms & Validation	React Hook Form + Zod	Create Role, Add Employee, Settings forms
Data Fetching / Cache	TanStack Query	Dashboard/analytics polling, optimistic updates on approve/reject actions
Charts	Recharts / Nivo	GMV trend, donuts (payment method, order status, settlement overview), growth charts
Command Palette	cmdk	The "⌘K" global search seen in every screen's top bar
Rich Diff/Notes	Tiptap or plain textarea with autosave	Admin Notes, Quick Notes on Organizer/Event Review

Backend

Layer	Choice	Why
API Framework	NestJS (Node.js, TypeScript) — shared platform backend	Admin-scoped modules (AdminAuth, RBAC, Moderation, Reporting) alongside existing Organizer/Customer domains
Auth	Email/password (argon2/bcrypt) + mandatory MFA recommended given financial/PII access	Matches Login screen; MFA is a strong addition given the stated 'monitored access' posture
Authorization	Centralized policy/guard layer driven by Role/RolePermission tables	Single enforcement point for the module × action permission matrix from §4.11
ORM / DB	Prisma + PostgreSQL (shared schema with Organizer/Customer backends)	Single source of truth for Event/Order/Settlement — Admin never holds a separate copy
Analytics / Reporting	A read-replica or dedicated warehouse (e.g. ClickHouse/BigQuery) fed by CDC/ETL from Postgres	Keeps heavy Analytics/Dashboard aggregation off the transactional database
Cache	Redis	Dashboard KPI caching, session storage, rate limiting on Admin Login
Background Jobs	BullMQ	Settlement generation, refund processing, scheduled analytics rollups, SLA-breach

Layer	Choice	Why
		alerting
Audit Logging	Append-only AuditLog table + optionally shipped to a log sink (e.g. CloudWatch/Datadog)	Required by the explicit 'all access is monitored and logged' notice on Login

Third-Party & Platform Integrations

Function	Recommended Provider(s)	Used In
Payments oversight	Razorpay Dashboard/API (read) + RazorpayX (payouts)	Orders, Settlements, Refunds
Email/SMS/WhatsApp (admin-initiated)	AWS SES/SendGrid, MSG91/Gupshup	Send Message to organizer, refund/approval notifications
Error Tracking	Sentry	Admin frontend + backend
BI / Reporting	Metabase or a custom warehouse-backed dashboard	Analytics, Download Full Report / Download Report
MFA	TOTP (e.g. otplib) or WebAuthn	Admin Login hardening
SSO (optional, future)	Google Workspace SSO / Okta	If Zordr's internal team grows and needs centralized identity

7. System Architecture (High Level)

- **Client tier** — Next.js Admin Portal (desktop-first), served over a VPN-restricted or IP-allowlisted domain distinct from the public Organizer/Customer domains, given its access to PII and financial data.
- **API tier** — Shared NestJS API; Admin-scoped endpoints live under their own module namespace, enforced by the centralized RBAC guard described in §4.11 and §6.
- **Moderation workflow** — The Event publish pipeline becomes: Organizer Portal writes Event.status = Submitted → EventReview record created → Admin Review screen → Approve sets Event.status = Published (visible to Customer App) / Send Back reopens for organizer edits / Reject terminates the submission with a reason.
- **Reporting tier** — A CDC/ETL pipeline replicates Order/Settlement/Event/User data into a reporting store for Dashboard and Analytics, so heavy aggregate queries never compete with live checkout traffic on the primary database.
- **Audit tier** — Every mutating admin action (approve/reject/suspend/refund/role-change/settings-change) writes to an append-only AuditLog, ideally via a database trigger or an interceptor at the API layer so it cannot be bypassed by a future code path.

8. API Design (Representative Endpoints)

Method	Endpoint	Description
POST	/admin/auth/login	Admin email/password login (+ MFA challenge if enabled)
GET	/admin/dashboard/summary	Platform-wide KPI cards, trends, recent activity, live events
GET	/admin/organizers	List/search/filter organizers
GET	/admin/organizers/:id	Organizer detail incl. documents, bank/GST, events, revenue, settlements, support, notes

Method	Endpoint	Description
POST	/admin/organizers/:id/approve	Approve organizer KYC application
POST	/admin/organizers/:id/suspend	Suspend an organizer (blocks publishing/payouts)
POST	/admin/organizers/:id/notes	Add an internal note
GET	/admin/events	Platform-wide event list/filter
GET	/admin/events/:id/review	Full event review payload + checklist state
POST	/admin/events/:id/approve	Approve & publish an event
POST	/admin/events/:id/send-back	Send back to organizer with notes
POST	/admin/events/:id/reject	Reject an event submission
GET	/admin/orders	Platform-wide order list/filter/export
POST	/admin/orders/bulk-action	Bulk refund/export/etc. on selected orders
GET	/admin/customers	Customer directory/filter
POST	/admin/customers/:id/block	Block/unblock a customer
GET	/admin/settlements	Platform-wide settlement ledger
POST	/admin/settlements/generate	Trigger settlement generation for a period
GET	/admin/refunds	Refund request queue
POST	/admin/refunds/:id/review	Approve/reject a refund request
GET	/admin/support/tickets	Cross-platform support ticket queue
GET	/admin/analytics/*	GMV/orders/customers/top-performers reporting endpoints
GET	/admin/employees	Internal staff directory
POST	/admin/employees/invite	Invite a new employee with a role assignment
GET	/admin/roles	List roles (system + custom)
POST	/admin/roles	Create a custom role with a permission matrix
PATCH	/admin/roles/:id/permissions	Update a role's module permissions
GET	/admin/settings	Platform settings bundle
PATCH	/admin/settings	Update platform settings

Every endpoint above additionally requires a role-permission check for (module, action) as configured in §4.11, and every mutating endpoint writes an AuditLog entry.

9. Business & Validation Rules ("Tech Rules")

9.1 Access & Security

- Admin accounts are provisioned only via the Employees → Add Employee flow (invite-based), never self-service signup.
- System roles (Super Admin, Event Manager, Operations Manager, Support Executive, Finance Executive, Marketing Executive) are protected from deletion; only their permission grants can be edited.
- Every login and every mutating action is written to AuditLog with actor, timestamp, IP, and a before/after diff where applicable, per the Login screen's stated monitoring policy.

9.2 Organizer Governance

- Organizer KYC status transitions (Pending → Approved/Rejected, Approved → Suspended) are admin-only actions; an organizer cannot self-approve or self-reinstate.
- Suspending or rejecting an organizer must immediately cascade: their events are unpublished/hidden from discovery, and any pending settlement to them should move to On Hold pending review.

9.3 Event Moderation

- An event cannot reach Published status without passing through EventReview with all checklist items satisfied and an explicit Approve & Publish action; Send Back and Reject both require a reason, which is relayed back to the organizer.
- The Customer App's public discovery/search index must only include events with Event.status = Published — Pending Review and Sent-Back events are never publicly queryable, even via direct link.
- Re-submission after Send Back re-enters the review queue as a new EventReview cycle, preserving the prior cycle's history for audit.

9.4 Orders, Settlements & Refunds

- The Admin Portal must never maintain a divergent copy of Order/Settlement data from the Organizer Portal or Customer App — all three read the same tables via role-scoped queries.
- The platform fee percentage used in any settlement calculation must be read from a single FeeConfig source; the current discrepancy between the Organizer-facing 2%+₹5/ticket disclosure and the Admin-facing 5% figure (\$4.7) must be resolved into one authoritative model before this module ships, since it directly affects money owed to organizers.
- Refund approval here must trigger the same idempotent gateway-refund workflow as an organizer- or attendee-initiated refund, and must adjust the affected settlement's Net Payout if that settlement period is still open.
- Bulk order actions (bulk refund, bulk export) on large selections must be processed asynchronously with a job-status indicator, not a blocking synchronous request.

9.5 Support & SLAs

- First-response and resolution SLA timers start at ticket creation and are computed server-side; approaching/breached SLAs should trigger internal alerts (e.g. Slack/email to the assigned team), not just a passive dashboard number.

9.6 RBAC Enforcement

- Every module's View/Create/Edit/Delete/Export actions are individually gated; an employee with only 'View' on Settlements, for example, must not be able to hit the Generate Settlement or Export endpoints even if they know the URL.
- Deleting a role that still has assigned employees should be blocked (or require reassigning those employees first) to avoid orphaned access states.

10. Security & Compliance

- This portal is the highest-privilege surface in the platform (PII, bank details, payment data, ability to publish/unpublish events and move money) — it should sit behind additional network-level protection (VPN/IP allowlist) beyond standard TLS, and MFA should be strongly considered even though not shown in the current screens.
- Field-level encryption for any bank/payout data viewed here mirrors the Organizer Portal's requirement (§10 of that document); admins should see masked account numbers by default with an explicit reveal-and-log action for full details.
- Full audit trail (per §9.1) is the primary compliance control for an internal tool with this level of access, and should be immutable (append-only, ideally shipped to a separate log store an admin cannot edit).

- Data-access minimization: roles like Marketing Executive or Support Executive should see only the customer/order fields relevant to their function, not full financial/bank detail, per the principle of least privilege embodied in the Create Role permission grid.
- Compliance alignment with India's DPDP Act for any customer/organizer PII processed here, and with RBI-adjacent expectations around payment-aggregation oversight given this portal's role in settlements and refunds.

11. Non-Functional Requirements

Category	Target
Availability	99.5%+ (internal tool — slightly more tolerant than customer-facing surfaces, but Settlements/Refunds actions should never be blocked for long)
Performance	List/table views (Orders, Customers, Events) < 2s for typical filtered queries even at tens of thousands of rows, via server-side pagination and indexing
Scalability	Analytics/Dashboard aggregation must scale independently of transactional load (see reporting-tier architecture, §7)
Auditability	100% of mutating admin actions logged; audit log itself should be tamper-evident and retained per financial record-keeping requirements
Access Control	Zero-trust default: new roles start with no permissions; every grant is explicit (as modeled in Create Role's checkbox grid)
Browser Support	Latest 2 versions of Chrome/Edge/Firefox on desktop (this portal is not designed mobile-first)

12. Suggested Development Phases

Phase	Focus	Key Deliverables
Phase 1	Foundation & RBAC	Admin auth, AuditLog infrastructure, Role/RolePermission model, Employees/Manage Roles/Create Role screens
Phase 2	Organizer Governance	Organizer list/detail, KYC approve/suspend/reject actions, documents & bank/GST review
Phase 3	Event Moderation	Event list, EventReview workflow (checklist, approve/send-back/reject), publish-gate integration with Customer App discovery
Phase 4	Commerce Oversight	Orders, Settlements (incl. FeeConfig reconciliation), Refunds screens and workflows
Phase 5	Customers & Support	Customer directory + block/unblock, Support ticketing + SLA monitoring
Phase 6	Analytics & Reporting	Reporting/warehouse pipeline, Dashboard, Analytics screen, report exports
Phase 7	Hardening & Launch	MFA, network restriction (VPN/IP allowlist), penetration testing, audit-log immutability review, DPDP/RBI compliance review

13. Appendix — Screen-to-Feature Traceability

Screen File	Module	Primary Backend Concern
1_AS_Login_Screen	Admin Auth	AdminEmployee auth, AuditLog

Screen File	Module	Primary Backend Concern
2_AS_Dashboard	Dashboard	Platform-wide aggregation/reporting service
3_AS_Organizer_Screen	Organizers	Organizer directory query/filter
4_AS_Organizer_Detail_Screen	Organizers	Organizer, PayoutAccount, OrganizerDocument, Settlement, OrganizerNote
5_AS_Events_Screen	Events	Event directory query/filter
6_AS_Event_Review_Screen	Events	EventReview, Event.status transition
7_AS_Orders_Screen	Orders	Order, Ticket (admin-scoped read + bulk actions)
8_AS_Customer_Screen	Customers	User, block/unblock state
9_AS_Settlements_Screen	Settlements	Settlement, FeeConfig
10_AS-Refunds_Screen	Refunds	Refund request, gateway refund workflow
11_AS_Support_Screen	Support	SupportTicket, SLA monitoring
12_AS_Analytics_Screen	Analytics	Reporting/warehouse layer
13_AS_Employee_Screen	Employees	AdminEmployee, Role assignment, invite flow
14_AS_Manage_Roles_Screen	RBAC	Role, RolePermission
15_AS_Create_Role_Screen	RBAC	Role, RolePermission (creation)
16_AS_Setting_Screen	Settings	Platform settings, plan/usage info

— End of Document —